Ch12 擷取網頁資料

Ch12 擷取網頁資料

- 使用Python 所提供模組可以直接擷取網頁中的資料，先介紹如何取得網頁，接著介紹JSON 與XML 檔案的讀取與分析，最後介紹網頁分析的第三方函式庫BeautifulSoup 的使用。

12-1 模組urllib.request、urllib.response 與requests

- 模組urllib.request 用於下載指定網址的網頁，程式碼較為複雜，第三方函式庫requests，也可以下載指定網址的網頁，程式碼較為簡潔。

模組urllib.request的重要函式

重要函式與說明	程式碼與執行結果
urlopen(url) 開啓 url 所指定的網頁，並回傳 urllib.response 物件。	import urllib.request as ur url='http://www.python.org' resp=ur.urlopen(url) print(resp)
	<http.client.HTTPResponse object at 0x02A92570>

模組urllib.response 的重要函式與屬性

重要函式與說明	程式碼與執行結果
read() 讀取物件 urllib.response 的所有資料，資料皆爲 byte，需要使用函式 decode 轉換成字串。	import urllib.request as ur url='https://www.python.org' resp=ur.urlopen(url) data=resp.read() print(data)
	b'<!doctype html>\n<!--[if lt IE 7]> <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">… <![endif]-->\n\n \n\n \n \n\n</body>\n</html>\n'

重要函式與說明	程式碼與執行結果
geturl() 讀取物件 urllib.response 的網址。	import urllib.request as ur url='https://www.python.org' resp=ur.urlopen(url) print(resp.geturl())
	https://www.python.org
getheader() 讀取物件 urllib.response 的網頁表頭。	import urllib.request as ur url='https://www.python.org' resp=ur.urlopen(url) print(resp.getheaders())
	[('Server', 'nginx'), ('Content-Type', 'text/html; charset=utf-8'),…, 'max-age=63072000; includeSubDomains')]

重要函式與說明	程式碼與執行結果
status 伺服器回傳的常見狀態碼，如下。 狀態碼 / 表示 2xx：成功獲得資料，例如：200 表示「OK」。 4xx：用戶端錯誤，例如：404 表示「找不到」 5xx：伺服器錯誤，例如：502 表示「閘道故障」	import urllib.request as ur url='https://www.python.org' resp=ur.urlopen(url) print(resp.status) 200

12-1 模組urllib.request、urllib.response 與requests

- 模組requests 為第三方函式庫，需要使用pip 進行安裝，若是Windows 作業系統
- 在「命令提示字元」程式下，使用指令「pip install requests」，會自動從網路下載安裝模組requests，模組requests 的重要函式如下。

模組requests 的重要函式

重要函式與說明	程式碼與執行結果
requests.get(url) 開啓 url 所指定的網頁。	import requests url = 'http://www.python.org' data = requests.get(url) print(data)
	顯示整個網頁的原始碼，因爲過長而省略。

12-1-1 使用模組urllib.request 下載網頁

- (ch12\12-1-1- 模組urllib.request.py)
- 使用模組urllib.request，下載網址為

行號	範例 ( ：ch12\12-1-1- 模組 urllib.request.py)
1	import urllib.request as ur
2	url='https://www.python.org'
3	resp=ur.urlopen(url)
4	print(resp.geturl())
5	print(resp.status)
6	print(resp.getheaders())
7	data=resp.read()
8	print(data)
9	print(data.decode())

程式解說

- 第1 行：匯入模組urllib.request，重新命名為ur。
- 第2 行：設定url 為「https://www.python.org」。
- 第3行：顯示模組ur的函式urlopen，以url為輸入，將回傳的urllib.response物件，指定物件名稱為resp。

- 第4 行：使用函式print 顯示物件resp的函式geturl(網址) 到螢幕上。
- 第5 行：使用函式print 顯示物件resp的屬性status(網頁狀態) 到螢幕上。

- 第6 行：使用函式print 顯示物件resp的函式getheaders(網頁表頭) 到螢幕上。
- 第7 行：顯示物件resp 的函式read，將回傳的網頁資料物件，指定物件名稱為data。

- 第8 行：使用函式print 將物件data的網頁原始碼，以byte方式顯示到螢幕上。
- 第9 行：使用函式decode 將物件data 的網頁原始碼byte 資料轉換成字串，最後使用函式print 將轉換後的字串顯示到螢幕上。

執行結果

```
https://www.python.org
200
[('Server', 'nginx'), ('Content-Type', 'text/html; charset=utf-8'),…, 'max-age=63072000;
includeSubDomains')]
b'<!doctype html>\n<!--[if lt IE 7]>   <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">…資料過長
省略…<![endif]-->\n\n    \n\n    \n    \n\n</body>\n</html>\n'
<!doctype html>
<!--[if lt IE 7]>   <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">   <![endif]-->
…資料過長省略…
</body>
</html>
```

12-1-2 使用函式庫requests 下載網頁

- (ch12\12-1-2-requests.py)
- 使用第三方函式庫requests，下載網址為「https://www.python.org」的網頁。

行號	範例 ( ：ch12\12-1-2-requests.py)
1	import requests
2	url = 'http://www.python.org'
3	data = requests.get(url)
4	print(data.encoding)
5	print(data.status_code)
6	print(data.headers)
7	print(data.text)

程式解說

- 第1 行：匯入模組requests。
- 第2 行：設定url 為「https://www.python.org」。
- 第3 行：顯示模組requests的函式get，以url 為輸入，將回傳物件命名為data。
- 第4 行：使用函式print 顯示物件data的屬性encoding(網頁編碼) 到螢幕上。

- 第5 行：使用函式print 顯示物件data的屬性status_code(網頁狀態) 到螢幕上。
- 第6 行：使用函式print 顯示物件data的屬性headers(網頁表頭) 到螢幕上。
- 第7 行：使用函式print 顯示物件data的屬性text(網頁內容) 到螢幕上。

12-1-2　使用函式庫requests 下載網頁(ch12\12-1-2-requests.py)

執行結果

```
utf-8
200
{'Connection': 'keep-alive', 'Public-Key-Pins': 'max-age=600;…', 'Date': 'Tue, 21 Jun 2016
06:01:20 GMT', 'Content-Length': '47462'}
<!doctype html>
<!--[if lt IE 7]>   <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">   <![endif]-->
…資料過長省略…
</body>
</html>
```

12-2 存取JSON

- 模組json 的常用函式

重要函式與說明	程式碼與執行結果
dumps(obj) 將 Python 的資料結構物件 obj 轉換成 JSON 字串。	import json dic = { 1: 'a',2: 'b',3: 'c'} js = json.dumps(dic) print(js)
	{"1": "a", "2": "b", "3": "c"}
loads(s) 將 JSON 字串 (s) 轉換成 Python 的資料結構物件。	import json dic = { 1: 'a',2: 'b',3: 'c'} js = json.dumps(dic) dic2 = json.loads(js) print(dic2)
	{'1': 'a', '2': 'b', '3': 'c'}

12-2-1　模組json 的使用 (ch12\12-2-1- 模組json.py)

- 使用模組json 將Python 資料結構轉換成JSON 格式，接著將JSON 格式轉換成Python 資料結構，JSON 格式如下，有點像是Python 的字典。
  - {"1": "a", "2": "b", "3": "c"}

行號	範例 (：ch12\12-2-1- 模組 json.py)	執行結果
1	import json	
2	dic = { 1: 'a',2: 'b',3: 'c'}	
3	js = json.dumps(dic)	{"1": "a", "2": "b", "3": "c"}
4	print(js)	{'3': 'c', '1': 'a', '2': 'b'}
5	dic2 = json.loads(js)	
6	print(dic2)	

12-2-1　模組json 的使用 (ch12\12-2-1- 模組json.py)

- 第1 行：匯入模組json。
- 第2 行：設定字典dic 為「1: 'a',2: 'b',3: 'c' 」。
- 第3 行：使用模組json 的函式dumps，以dic 為輸入，將Python 資料結構轉換成JSON 格式，轉換後的JSON 物件名稱為js。

- 第4 行：使用函式print 顯示物件js到螢幕上。
- 第5 行：使用模組json 的函式loads，以js為輸入，將JSON 格式轉換成Python 資料結構，轉換後的Python 物件名稱為dic2。
- 第6 行：使用函式print 顯示物件dic2到螢幕上。

12-2-2　讀取全台PM2.5 測站資料

- 使用政府資料開放平台(https://data.gov.tw) 下載全台PM2.5 測站資料，會以JSON格式回傳，經由Python分析JSON 格式資料，獲得全台PM2.5 測站資料。

行號	範例 ( ：ch12\12-2-2- 讀取全台 PM2.5 測站資料 .py)
1	import requests
2	import json
3	url="https://opendata.epa.gov.tw/ws/Data/ATM00625/?$format=json"
4	result=requests.get(url, verify=False)
5	data=json.loads(result.text)
6	for item in data:
7	print(item['county'],item['Site'],item['PM25'],item['DataCreationDate'])

執行結果

```
新北市 富貴角 13 2018-11-12 20:00
雲林縣 麥寮 36 2018-11-12 20:00
臺東縣 關山 11 2018-11-12 20:00
澎湖縣 馬公 31 2018-11-12 20:00
金門縣 金門 20 2018-11-12 20:00
```

程式解說

- **第1 行**：匯入模組requests。
- **第2 行**：匯入模組json。
- **第3行**：設定url為「https://opendata.epa.gov.tw/ws/Data/ATM00625/?$format=json」。
- **第4 行**：顯示模組requests 的函式get，以url 為輸入，設定verify 為False 表示忽略https 認證，將回傳物件命名為result。

- **第5 行**：使用模組json 的函式loads，以物件result 的屬性text 為輸入，將JSON 格式轉換成Python 資料結構，轉換後的Python 物件名稱為data。
- **第6 到7 行**：外層使用for 迴圈找出data 的所有資料，依序指定給變數item，item['county'] 表示縣市名稱，item['Site'] 表示地點，item['PM25'] 表示PM2.5 的數值，item['DataCreationDate'] 表示量測日期與時間。

12-3 存取XML

- XML 是一種標記的語法，使用標籤來標記資料，標籤可以嵌套於其他標籤內，可以用於資訊的傳遞
- 例如：網站提供最新消息的RSS 功能就是一種XML 格式。以下為XML 格式範例，此XML 檔存放在資料夾ch12 檔案名稱為my.xml。

```
<class>
  <morning time="8:00-12:00"> 上午課程
    <item time="8:10-10:00" where=" 電腦教室 "> 電腦 </item>
    <item time="10:10-12:00"> 英文 </item>
  </morning>
  <afternonn time="13:00-17:00"> 下午課程
    <item time="13:10-14:00" where=" 操場 "> 體育 </item>
    <item time="14:10-16:00"> 數學 </item>
    <item time="16:10-17:00"> 地理 </item>
  </afternonn>
</class>
```

說明

- 1. 標籤class 內有標籤morning 與標籤afternoon，標籤morning 與標籤afternoon 內還有標籤item。
- 2. time 與where 是item 的「屬性」。
- 3. 標籤內可以擺放「值」，例如下午第一節課為「體育」，「體育」為下午第一節課標籤item 的「值」。

12-3　存取XML

- Python 提供模組xml.etree.ElementTree 分析XML 格式的資料，以下介紹模組 xml.etree.ElementTree 的常用函式。

模組
xml.etree.ElementTree 的常用函式

重要函式與說明	程式碼與執行結果
ElementTree.ElementTree(xml_file) 將 xml_file 轉換成 ElementTree，回傳 ElementTree 物件。	import xml.etree.ElementTree as xmltree tree = xmltree.ElementTree(file='my.xml') 註：my.xml 爲本節的 XML 格式範例。
	執行結果： 匯入 my.xml 檔到 ElementTree，獲得 ElementTree 物件指定給物件 tree。

重要函式與說明	程式碼與執行結果
ElementTree.ElementTree(xml_file).getroot() ElementTree 物件的函式 getroot 會回傳 Element 物件。	import xml.etree.ElementTree as xmltree tree = xmltree.ElementTree(file='my.xml') root = tree.getroot() print(root.tag) 註：my.xml 爲本節的 XML 格式範例。
	執行結果： class

<table>
<tr><td rowspan="2">ElementTree.fromstring(xml_string)
將 xml_string 轉換成 Element，回傳物件 Element。</td><td>import requests
import xml.etree.ElementTree as xmltree
url="https://pypi.python.org/pypi?%3Aaction=rss"
result=requests.get(url)
element=xmltree.fromstring(result.text)</td></tr>
<tr><td>執行結果：
從網址 https://pypi.python.org/pypi?%3Aaction=rss，讀取 RSS 檔，轉換成字串 result.text，經由函式 fromstring 轉換成物件 Element。</td></tr>
</table>

ElementTree.dump(element)
將 Element 物件轉成 XML 格式。

```
import requests
import xml.etree.ElementTree as xmltree
url="https://pypi.python.org/pypi?%3Aaction=rss"
result=requests.get(url)
element=xmltree.fromstring(result.text)
print(xmltree.dump(element))
```

執行結果：
從網址 https://pypi.python.org/pypi?%3Aaction=rss，讀取 RSS 檔，轉換成字串 result.text，經由函式 fromstring 轉換成物件 Element，最後經由函式 dump 將物件 Element 轉換成 XML 格式。

模組xml.etree.Element 的常用函式

重要函式與說明	程式碼與執行結果
ElementTree.Element.iter(tag) 回傳 Element 物件中標籤爲 tag 的所有標籤。	import xml.etree.ElementTree as xmltree tree = xmltree.ElementTree(file='my.xml') root = tree.getroot() for item in root.iter('item'): print(item.attrib , item.text) 註：my.xml 爲本節的 XML 格式範例。
	執行結果： {'time': '8:10-10:00', 'where': ' 電腦教室 '} 電腦 {'time': '10:10-12:00'} 英文 {'time': '13:10-14:00', 'where': ' 操場 '} 體育 {'time': '14:10-16:00'} 數學 {'time': '16:10-17:00'} 地理

ElementTree.Element.findall(match) 找出符合 match 所指定標籤路徑的所有標籤。	import xml.etree.ElementTree as xmltree tree = xmltree.ElementTree(file='my.xml') root = tree.getroot() for item in root.findall('./morning/item'): print(' 標籤爲 ', item.tag, ' ，屬性 ', item.attrib, ' ，值 ', item.text)
	執行結果： 標籤爲 item ，屬性 {'where': ' 電腦教室 ', 'time': '8:10-10:00'} ，值 電腦 標籤爲 item ，屬性 {'time': '10:10-12:00'} ，值 英文

12-3-1　使用模組 xml.etree.ElementTree 分析XML

- (ch12\12-3-1- 分析XML.py)
- 使用模組xml.etree.ElementTree 讀取XML 檔案，列出XML 檔案中的標籤、屬性與值。

行號	範例（檔案名稱：ch12\12-3-1- 分析 XML.py）
1	import xml.etree.ElementTree as xmltree
2	tree = xmltree.ElementTree(file='my.xml')
3	root = tree.getroot()
4	print(root.tag)
5	for a in root:
6	print(' 標籤 ', a.tag, ' ，屬性 ', a.attrib, ' ，值 ', a.text)
7	for b in a:
8	print(' 標籤 ', b.tag, ' ，屬性 ', b.attrib, ' ，值 ', b.text)
9	for item in root.iter('item'):
10	print(item.attrib , item.text)
11	for item in root.findall('./morning/item'):
12	print(' 標籤爲 ', item.tag, ' ，屬性 ', item.attrib, ' ，值 ', item.text)

程式解說

- 第1 行：匯入模組xml.etree.ElementTree，重新命名為xmltree。
- 第2 行：使用模組xmltree 的函式ElementTree，以檔案my.xml 為輸入，將回傳的物件ElementTree 指定給物件tree。

- 第3 行：執行物件tree 的函式getroot，回傳物件Element指定給物件root。
- 第4 行：使用函式print 顯示物件root 的屬性tag。

- 第5 到8 行：使用巢狀迴圈，顯示物件root 的往下兩層的標籤、屬性與值。
- 外層迴圈使用for 迴圈取出物件root 的下一層標籤到物件a，顯示物件a 的標籤、屬性與值到螢幕上(第6 行)
- 內層迴圈使用for 迴圈取出物件a 的下一層標籤到物件b，顯示物件b 的標籤、屬性與值到螢幕上(第8 行)。

- 第9到10行：使用物件root的函式iter，以標籤名稱「item」，找出所有標籤為「item」的標籤，接著使用for 迴圈將這些標籤指定給物件item，顯示物件item 的屬性與值到螢幕上。

- 第11 到12 行：使用物件root 的函式findall，以路徑「./morning/item」，找出物件root 的下一層標籤為「morning」中的下一層標籤為「item」的所有標籤，接著使用for 迴圈將這些標籤指定給物件item，顯示物件item 的標籤、屬性與值到螢幕上。

12-3-1 使用模組 xml.etree.ElementTree 分析XML

執行結果

```
class
標籤 morning ，屬性 {'time': '8:00-12:00'} ，值 上午課程
標籤 item ，屬性 {'time': '8:10-10:00', 'where': ' 電腦教室 '} ，值 電腦
標籤 item ，屬性 {'time': '10:10-12:00'} ，值 英文
標籤 afternoon ，屬性 {'time': '13:00-17:00'} ，值 下午課程
標籤 item ，屬性 {'time': '13:10-14:00', 'where': ' 操場 '} ，值 體育
標籤 item ，屬性 {'time': '14:10-16:00'} ，值 數學
標籤 item ，屬性 {'time': '16:10-17:00'} ，值 地理
{'time': '8:10-10:00', 'where': ' 電腦教室 '} 電腦
{'time': '10:10-12:00'} 英文
{'time': '13:10-14:00', 'where': ' 操場 '} 體育
{'time': '14:10-16:00'} 數學
{'time': '16:10-17:00'} 地理
標籤爲 item ，屬性 {'time': '8:10-10:00', 'where': ' 電腦教室 '} ，值 電腦
標籤爲 item ，屬性 {'time': '10:10-12:00'} ，值 英文
```

12-3-2 從PyPI 網站讀取最後更新50 個套件的RSS

- (ch12\12-3-2- 讀取RSS.py)
- 使用模組xml.etree.ElementTree 讀取PyPI 網站最後更新50 個套件的RSS 檔，列出RSS 檔中指定標籤的標籤名稱與對應的值。

行號	範例 ( ：ch12\12-3-2- 讀取 RSS.py)
1	import requests
2	import xml.etree.ElementTree as xmltree
3	url="https://pypi.python.org/pypi?%3Aaction=rss"
4	result=requests.get(url)
5	element=xmltree.fromstring(result.text)
6	print(xmltree.dump(element))
7	for item in element.findall('./channel/item'):
8	for b in item:
9	print(b.tag, b.text)
10	print()

程式解說

- 第1 行：匯入模組requests。
- 第2 行：匯入模組xml.etree.ElementTree，重新命名為xmltree。
- 第3 行：設定url 為「https://pypi.python.org/pypi?%3Aaction=rss」，此為PyPI 網站最後更新50 個套件的RSS 網址。

- 第4 行：使用模組requests的函式get，以url 為輸入，將回傳物件命名為result。
- 第5行：使用模組xmltree的函式fromstring，以檔案物件result的屬性text為輸入，將回傳的物件Element 指定給物件element。
- 第6 行：使用模組xmltree 的函式dump，以物件element 為輸入，將結果使用函式print 顯示在螢幕上。

- 第7 到10 行：使用物件element 的函式findall，以路徑「./channel/item」，找出物件element 的下一層標籤為「channel」中的下一層標籤為「item」的所有標籤
- 接著外層迴圈使用for 迴圈將這些標籤指定給物件item，內層迴圈使用for 迴圈將物件item 下一層標籤指定給物件b，顯示物件b的標籤與值到螢幕上(第9 行)，內層迴圈結束後，使用函式print 進行換行(第10 行)。

12-3-2 從PyPI 網站讀取最後更新50 個套件的RSS (ch12\12-3-2- 讀取RSS.py)

執行結果

```
<rss version="0.91">
  <channel>
    <title>PyPI recent updates</title>
    …資料過長省略…
    <item>
      <title>dmpdfparserdata 1.0.2</title>
      <link>https://pypi.org/project/dmpdfparserdata/1.0.2/</link>
      <description>dmpdfparserdata</description>
      <pubDate>Fri, 04 May 2018 08:16:05 GMT</pubDate>
    </item>
    …資料過長省略…
  </channel>
</rss>
title dmpdfparserdata 1.0.2
link https://pypi.org/project/dmpdfparserdata/1.0.2/
description dmpdfparserdata
pubDate Fri, 04 May 2018 08:16:05 GMT
…資料過長省略…
```

12-4 使用套件Beautiful Soup 存取HTML

- HTML(HyperText Markup Language) 是一種標記語言，瀏覽器經由讀取標籤與資料，使用標籤決定資料顯示在網頁對應的功能與位置上。
- 以下為HTML 格式範例，此HTML 檔存放在資料夾ch12 檔案名稱為web.htm。

12-4 使用套件Beautiful Soup 存取HTML

```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>Python 相關網站 </title>
<style type="text/css">
<!--
.table_head {
        font-family: Arial, Helvetica, sans-serif;
        font-size: 24px;
        color: #0000FF;
        background-color: #FFFF33;
}
.table_sitename {
        font-family: "Times New Roman", Times, serif;
        font-size: 22px;
        color: #FF0000;
        background-color: #CCCCCC;
}
```

```
.table_siteurl {
        font-family: "Times New Roman", Times, serif;
        color: #0000FF;
        background-color: #CCCCCC;
        font-size: 22px;
}
.style1 {
        font-size: x-large
}
-->
</style>
</head>
<body>
<p class="style1">Python 相關網站 </p>
<table width="732" border="1">
  <tr>
    <td width="205" class="table_head"> 網站名稱 </td>
    <td width="511" class="table_head"> 網址 </td>
```

```
  </tr>
  <tr>
    <td class="table_sitename">Python</td>
    <td class="table_siteurl"><a href="https://www.python.org/">https://www.python.org/</
a></td>
  </tr>
  <tr>
    <td class="table_sitename">PyPI</td>
    <td class="table_siteurl"><a href="https://pypi.python.org/pypi">https://pypi.python.
org/pypi</a></td>
  </tr>
</table>
</body>
</html>
```

說明

- 標籤以「< 標籤名稱></ 標籤名稱>」表示標籤的範圍，以「< 標籤名稱>」為開始，「</ 標籤名稱>」為結束。
- 例如：HTML 格式的最外層為 <html></html>，表示HTML 的開始與結束，標籤<html></html> 內分成 <head></head> 與<body></body>兩部分。

模組
BeautifulSoup 的重要函式

重要函式與說明	程式碼與執行結果
BeautifulSoup(html, 'html.parser') 參數 html 爲 HTML 格式的字串，輸入到模組 BeautifulSoup，轉換成 Python 物件。	from bs4 import BeautifulSoup as soup fin = open('web.htm', encoding='utf-8') s = fin.read() htm = soup(s, 'html.parser') 註：web.htm 爲本節的 HTML 範例。
	執行結果： 讀取檔案 web.htm 的內容經由 BeautifulSoup 轉換成 Python 物件。

重要函式與說明	程式碼與執行結果

	from bs4 import BeautifulSoup as soup
	fin = open('web.htm', encoding='utf-8')
	s = fin.read()
	htm = soup(s, 'html.parser')
	for item in htm.find_all('tr'):
	  print(item)
	for item in htm.find_all('td',class_='table_head'):
	  print(item)
	註：web.htm 爲本節的 HTML 範例。

BeautifulSoup.find_all(tag) BeautifulSoup.find_all(tag, attr) 從 HTML 找出標籤爲「tag」的所有元素。若參數 attr 存在，則從 HTML 找出標籤爲「tag」的所有元素，且屬性也要符合「attr」的要求。	執行結果： <tr> <td class="table_head" width="205"> 網站名稱 </td> <td class="table_head" width="511"> 網址 </td> </tr> <tr> <td class="table_sitename">Python</td> <td class="table_siteurl">https://www.python.org/</td> </tr> <tr> <td class="table_sitename">PyPI</td> <td class="table_siteurl">https://pypi.python.org/pypi</td> </tr> <td class="table_head" width="205"> 網站名稱 </td> <td class="table_head" width="511"> 網址 </td>

12-4　使用套件Beautiful Soup 存取HTML

重要函式與說明	程式碼與執行結果
BeautifulSoup.prettify() 將 HTML 資料轉換成較容易閱讀的方式呈現。	from bs4 import BeautifulSoup as soup fin = open('web.htm', encoding='utf-8') s = fin.read() htm = soup(s, 'html.parser') print(htm.title.prettify()) 註：web.htm 爲本節的 HTML 範例。
	執行結果： <title> Python 相關網站 </title>

模組BeautifulSoup 的重要屬性

重要屬性與說明	程式碼與執行結果
tag.contents 找出第一個遇到標籤爲「tag」所對應的值，所有值都會加入串列中，回傳此串列。	from bs4 import BeautifulSoup as soup fin = open('web.htm', encoding='utf-8') s = fin.read() htm = soup(s, 'html.parser') print(htm.title.contents) print(htm.title.contents[0]) 註：web.htm 爲本節的 HTML 範例。
	執行結果： ['Python 相關網站 '] Python 相關網站

重要屬性與說明	程式碼與執行結果
tag.name 找出第一個遇到標籤爲「tag」所對應的標籤名稱。	from bs4 import BeautifulSoup as soup fin = open('web.htm', encoding='utf-8') s = fin.read() htm = soup(s, 'html.parser') print(htm.title.name) 註：web.htm 爲本節的 HTML 範例。
	執行結果： title

<table>
<tr><th>重要屬性與說明</th><th>程式碼與執行結果</th></tr>
<tr><td rowspan="2">tag.string
找出第一個遇到標籤爲「tag」，當該標籤只有一個值，且爲字串或數值，則回傳該值。</td><td>from bs4 import BeautifulSoup as soup
fin = open('web.htm', encoding='utf-8')
s = fin.read()
htm = soup(s, 'html.parser')
print(htm.title.string)</td></tr>
<tr><td>執行結果：
Python 相關網站</td></tr>
</table>

重要屬性與說明	程式碼與執行結果
tag['attr'] 找出第一個遇到標籤爲「tag」，且該標籤有屬性「attr」，則回傳該屬性所對應的值。	from bs4 import BeautifulSoup as soup fin = open('web.htm', encoding='utf-8') s = fin.read() htm = soup(s, 'html.parser') print(htm.head.meta) print(htm.head.meta['content']) 執行結果： <meta content="text/html; charset=utf-8" http-equiv="Content-Type"/> text/html; charset=utf-8

12-4-1　使用模組BeautifulSoup分析HTML

- (ch12\12-4-1- 模組BeautifulSoup.py)
- 使用模組BeautifulSoup 讀取HTML 檔案，找出HTML 檔案指定標籤的名稱、屬性或值，本範例的HTML 檔放在此範例程式資料夾下，檔案名稱為「web.htm」。

範例 (: ch12\12-4-1- 模組 BeautifulSoup.py)

```
1  from bs4 import BeautifulSoup as soup
2  fin = open('web.htm', encoding='utf-8')
3  s = fin.read()
4  htm = soup(s, 'html.parser')
5  print(htm.title.prettify())
6  print(htm.title.contents)
7  print(htm.title.contents[0])
8  print(htm.title.name)
9  print(htm.title.string)
10 print(htm.meta)
11 print(htm.meta['content'])
12 for item in htm.find_all('td'):
13     print(item)
14 for item in htm.find_all('td',class_='table_head'):
15     print(item)
16 for item in htm.find_all('td',class_='table_siteurl'):
17     print(item.a['href'])
```

程式解說

- 第1 行：從套件bs4匯入模組BeautifulSoup，重新命名為soup。
- 第2 行：開啟檔案web.htm，以utf-8 為文字編碼，指定給物件fin。
- 第3 行：使用物件fin 的函式read 讀取檔案內容，指定給物件s。

- 第4 行：使用模組soup，以物件s 與「html.parser」，轉換成Python 物件，指定給物件htm。
- 第5 行：使用物件htm 下，找到第一個標籤「title」，使用函式prettify，轉換成較容易閱讀的方式呈現。

- 第6行：使用物件htm下，找到第一個標籤「title」的屬性contents，將標籤「title」所包夾的所有子標籤元素，記錄在屬性contents。
- 第7 行：使用物件htm 下，找到第一個標籤「title」的屬性contents[0]，取出標籤「title」所包夾的子標籤串列的第1 個元素。

- 第8 行：使用物件htm 下，找到第一個標籤「title」的屬性name(標籤名稱)。
- 第9 行：使用物件htm 下，找到第一個標籤「title」的屬性string(標籤對應值)。

- 第10 行：使用物件htm 下，找到第一個標籤「meta」，使用函式print 將結果顯示在螢幕上。
- 第11 行：使用物件htm 下，找到第一個標籤「meta」屬性「content」，使用函式print 將結果顯示在螢幕上。

- 第12 到13 行：從物件htm 找出標籤為「td」的所有元素，依序取出這些元素指定給物件item，使用函式print 將物件item 顯示在螢幕上。

- 第14 到15 行：從物件htm 找出標籤為「td」
- 且屬性「class」等於「table_head」的所有元素，依序取出這些元素指定給物件item，使用函式print 將物件item 顯示在螢幕上。

- 第16 到17 行：從物件htm 找出標籤為「td」
- 且屬性「class」等於「table_siteurl」的所有元素，依序取出這些元素指定給物件item，使用函式print 將物件item 的標籤「a」的屬性「href」顯示在螢幕上。

執行結果

```
<title>
 Python 相關網站
</title>

['Python 相關網站 ']
Python 相關網站
title
Python 相關網站
<meta content="text/html; charset=utf-8" http-equiv="Content-Type"/>
text/html; charset=utf-8
<td class="table_head" width="205"> 網站名稱 </td>
<td class="table_head" width="511"> 網址 </td>
<td class="table_sitename">Python</td>
```

```
<td class="table_siteurl"><a href="https://www.python.org/">https://www.python.org/</
a></td>
<td class="table_sitename">PyPI</td>
<td class="table_siteurl"><a href="https://pypi.python.org/pypi">https://pypi.python.org/
pypi</a></td>
<td class="table_head" width="205"> 網站名稱 </td>
<td class="table_head" width="511"> 網址 </td>
https://www.python.org/
https://pypi.python.org/pypi
```

12-4-2 使用模組Beautiful Soup找出網頁中所有超連結網址

- (ch12\12-4-2- 使用BeautifulSoup 找出所有超連結.py)
- 使用模組requests 讀取網址中的網頁，接著使用模組Beautiful Soup，找出網頁中的所有超連結網址。

行號	範例 ( ：ch12\12-4-2- 使用 BeautifulSoup 找出所有超連結 .py)
1	import requests
2	from bs4 import BeautifulSoup as soup
3	url='http://www.python.org'
4	def h(url):
5	if url[:4] == 'http' or url[:5]=='https':
6	return True
7	def links(url):
8	page = requests.get(url).text
9	htm=soup(page,'html.parser')
10	alinks=[item['href'] for item in htm.find_all('a')]
11	links=[x for x in alinks if h(x)]
12	return links
13	print(' 找出網址爲 ',url,' 的 http 與 https 開頭的超連結 ')
14	for link in links(url):
15	print(link)

程式解說

- **第1 行**：匯入模組requests。
- **第2 行**：從套件bs4匯入模組BeautifulSoup，重新命名為soup。
- **第3 行**：設定url 為「http://www.python.org」。
- **第4 到6 行**：定義函式h，輸入參數url，若url 開頭為「http」或「https」，則回傳True。

- 第7 到12 行：定義函式links，輸入參數url，使用模組requests 的函式get，以url為輸入，取回傳物件的屬性text，指定給物件page(第8 行)。
- 使用模組soup，以物件page 與「html.parser」，轉換成Python 物件，指定給物件htm(第9 行)。

- 使用串列生成式，使用物件htm 的函式find_all，找出標籤為「a」的所有物件，依序指定給物件item，取出物件item 的屬性「href」加入到串列alinks(第10 行)，使用串列生成式，串列alinks 的所有物件，依序指定給物件x，使用函式h 以物件x 為輸入，若結果為True，則加入到串列links(第11 行)，回傳物件links(第12 行)。

- 第13 行：使用函式print 顯示「找出網址為」，串接物件url，串接「的http 與https 開頭的超連結」到螢幕上。
- 第14 到15 行：呼叫函式links，以url 為輸入，將回傳的串列使用for 迴圈依序取出到物件link，接著使用函式print 顯示物件link 到螢幕上。

執行結果

找出網址爲 http://www.python.org 的 http 與 https 開頭的超連結

https://docs.python.org

https://pypi.python.org/

…資料過長省略…

https://github.com/python/pythondotorg/issues

https://status.python.org/

12-4-3　使用模組Beautiful Soup找出Python 網站的最新消息

- (ch12\12-4-3- 使用BeautifulSoup 找出Python 網站的最新消息.py)
- 使用模組requests 讀取Python 網站(http://www.python.org/) 的首頁，接著使用模組Beautiful Soup，找出首頁中最新消息，使用瀏覽器瀏覽http://www.python.org/，接著檢視網頁原始碼後，發現最新消息區塊的HTML 網頁如下。

```
1  <div class="shrubbery">
2    <h2 class="widget-title"><span aria-hidden="true" class="icon-news"></
   span>Latest News</h2>
3    <p class="give-me-more"><a href="http://blog.python.org" title="More
4  News">More</a></p>
5    <ul class="menu">
6     <li><time datetime="2016-06-14T06:06:00.000001+00:00"><span class="say-
   no-more">2016-</span>06-14</time><a href="http://feedproxy.google.com/~r/
   PythonInsider/~3/yaupkpp9kLc/python-2712-release-candidate-available.
   html">The first release candidate of Python 2.7.12, the next bugfix ...</a></li>
```

```
7       <li><time datetime="2016-06-14T04:06:00.000001+00:00"><span class="say-
        no-more">2016-</span>06-14</time><a href="http://feedproxy.google.com/~r/
        PythonInsider/~3/nE_a_L4mHfk/python-360-alpha-2-preview-release-is.
        html">Python 3.6.0a2 has been released.  3.6.0a2 is the
        second of four planned alpha ...</a></li>
8       <li><time datetime="2016-06-13T03:18:00+00:00"><span class="say-no-
        more">2016-</span>06-13</time><a href="http://feedproxy.google.com/~r/
        PythonInsider/~3/sDMmUdw_7RU/python-352rc1-and-python-345rc1-are-now.
        html">Python 3.5.2rc1 and Python 3.4.5rc1 are now available for download. ...</
        a></li>
9       <li><time datetime="2016-05-17T22:57:00.000001+00:00"><span class="say-
        no-more">2016-</span>05-17</time><a href="http://feedproxy.google.com/~r/
        PythonInsider/~3/cxjZlK_6b9Y/python-360-alpha-1-preview-release-is.html">Python
        3.6.0a1 has been released.  3.6.0a1 is the first of four
        planned alpha ...</a></li>
```

```
10      <li><time datetime="2015-12-07T05:51:00+00:00"><span class="say-no-
more">2015-</span>12-07</time><a href="http://feedproxy.google.com/~r/
PythonInsider/~3/F9ApGB7BGB4/python-351-and-python-344rc1-are-now.
html">Python 3.5.1 and Python 3.4.4rc1 are now available for download. ...</a></
li>
11    </ul>
</div><!-- end .shrubbery -->
```

12-4-3 使用模組Beautiful Soup找出Python 網站的最新消息

行號	範例 (: ch12\12-4-3- 使用 BeautifulSoup 找出 Python 網站的最新消息 .py)
1	import requests
2	from bs4 import BeautifulSoup as soup
3	url='http://www.python.org/'
4	def getnews(url):
5	page = requests.get(url).text
6	doc=soup(page,'html.parser')
7	items = [elem for elem in doc.find_all('div', class_='shrubbery')]
8	for item in items:
9	if item.h2.contents[1] == 'Latest News':
10	ys = [y for y in item.find_all('li')]
11	for y in ys:
12	time = y.time['datetime']
13	link = y.a['href']
14	title = y.a.string
15	print(time, title, link)
16	getnews(url)

程式解說

- **第1 行**：匯入模組requests。
- **第2 行**：從套件bs4匯入模組BeautifulSoup，重新命名為soup。
- **第3 行**：設定url 為「http://www.python.org」。

- **第4 到15 行**：定義函式getnews，輸入參數url，使用模組requests 的函式get，以url 為輸入，取回傳物件的屬性text，指定給物件page(第5 行)。使用模組soup，以物件page與「html.parser」，轉換成Python 物件，指定給物件doc(第6 行)。
- 使用串列生成式，使用物件doc 的函式find_all，找出標籤為「div」且屬性class 等於「shrubbery」的所有物件，指定此串列給物件items(第7 行)。

- 第8 到15 行：使用for 迴圈依序取出串列items 的元素到物件item，若物件item中標籤「h2」的標籤值的第2 個元素等於「Latest News」，則使用串列生成式以物件item 的函式find_all，找出標籤為「li」的所有物件加入串列，指定此串列給物件ys(第10 行)

- 使用for 迴圈依序取出串列ys 的元素到物件y，取出物件y 中標籤「time」的屬性「datetime」指定給物件time(第12 行)，取出物件y 中標籤「a」的屬性「href」指定給物件link(第13 行)，取出物件y 中標籤「a」的值指定給物件title(第14 行)，使用函式print 顯示物件time、title 與link 到螢幕上(第15 行)。
- 第16 行：呼叫函式getnews，以url 為輸入。

執行結果

```
2016-06-14T06:06:00.000001+00:00 The first release candidate of Python 2.7.12, the
next bugfix ... http://feedproxy.google.com/~r/PythonInsider/~3/yaupkpp9kLc/python-
2712-release-candidate-available.html
…資料過長省略 ..
2015-12-07T05:51:00+00:00 Python 3.5.1 and Python 3.4.4rc1 are now available for
download. ... http://feedproxy.google.com/~r/PythonInsider/~3/F9ApGB7BGB4/python-
351-and-python-344rc1-are-now.html
```